

```
1 #ifndef MAP_H
2 #define MAP_H
3
4 #include <assert.h>
5 #include <stdbool.h>
6 #include <stdio.h>
7 #include <stdlib.h>
8 #include "linked_list.h"
9 #include "group.h"
10
11 typedef unsigned long long llong;
12
13 typedef struct PermIterator {
14     int* data;
15     int n;
16     int finished;
17 } PermIterator_t;
18
19 int min(int a, int b);
20
21 // Random permutation utilities
22 void random_shuffle_array(int* arr, int n);
23 int* get_random_subset(int m, int n);
24 int* get_random_permutation(int m);
25
26 // Calculations.
27 llong factorial(int n);
28 long long n_choose_k(int n, int k);
29 int** get_binomial(int n);
30 void free_matrix(int** matrix, int n);
31 void print_matrix(int** matrix, int n);
32
33 // QOF
34 void print_map(int* map, int n);
35 void print_arr(int* arr, int n);
36 int* get_inverse_perm(int* p, int n);
37
38 // Subsets.
39 bool* init_subset(int m, int n);
40 bool is_last_subset(bool* subset, int m, int n);
41
42 void push_to_subset(list_t* sub, int i, int v, int k, int n, int** binom);
43 list_t* get_subset_list_from_binom(int i, int m, int n, int** binom);
44 list_t* get_subset_list(int i, int m, int n);
45 int* array_from_list(list_t* l);
46 int* map_from_list(list_t* image);
47 int* get_array_from_subset_binom(int i, int m, int n, int** binom);
48
49 void map_image_generators(int* map, list_t* gen, int* image);
50 void map_elements(int* map, list_t* l1, list_t* l2);
51 int* get_full_map(gdata_t* g1_data, group_t* g2, int* image_gen, int m);
52 int* get_induced_map(gdata_t* g1_data, group_t* g2, list_t* image_gen, int m);
53
54 void rec_generate_permutation_into(int n, llong k, int *res);
55 int* get_kth_permutation_rec(int n, llong k);
56 void apply_permutation(int* perm, int* array, int n);
57 int* get_from_permutation(int* perm, int* array, int n);
58
59 void get_next_subset(bool* set, int m, int n);
60 int* subset_to_map(bool* set, int m, int n);
```

```
61 bool* map_to_subset(int* map, int m, int n);
62
63 int* free_get_next_map(int* map, int m, int n);
64 int* get_next_map(int* map, int m, int n);
65 int calculate_from_list(group_t* g, list_t* l);
66
67 // Return:  $h(x) = h(g_1 \rightarrow \dots \rightarrow g_n) = h(g_1)h(g_2)\dots h(g_n)$  with  $h$  partial map of
generators
68 int get_extension_value(group_t* g2, list_t* l, int* partial_map);
69
70 //  $m = |\text{image}|$ 
71 int* get_induced_hom(group_t* g2, gdata_t* data, int* image, int m);
72
73 bool is_isomorphism(group_t* g1, gdata_t* gdata, group_t* g2, int* map, int n);
74 bool is_isomorphism_with_order(group_t* g1, gdata_t* gdata, group_t* g2, int*
orders_g1, int* orders_g2, int* map, int n);
75 bool next_injective_map(int *map, bool *used, int m, int n);
76
77 #endif
```